이지훈 Android Developer Portfolio

Phone 010-2010-3068	**Email** mraz3068@gmail.com
GitHub github.com/easyhooon	**Tech Blog** velog.io/@mraz3068
Resume RESUME.md	**LinkedIn** linkedin.com/in/easyhooon

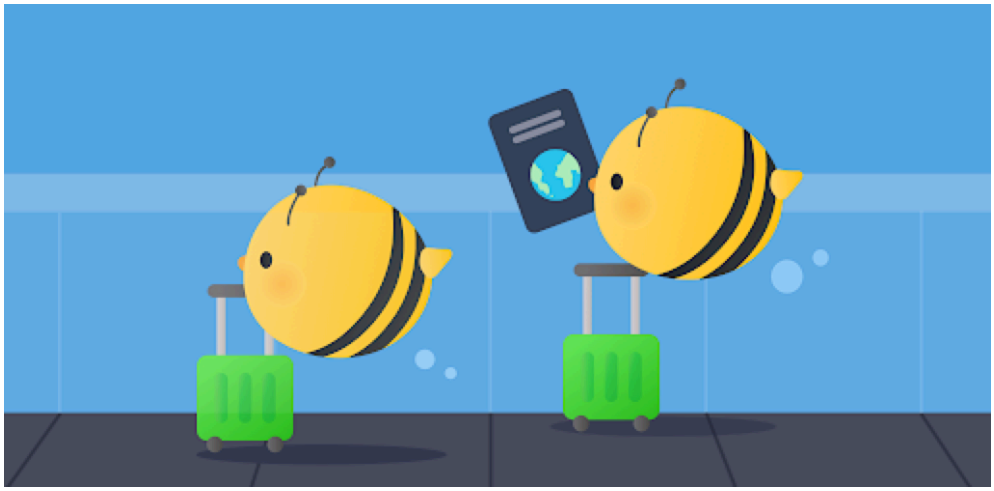
Overview

출시한 **Android 앱 9개와 iOS 앱 1개**를 중심으로, 제품을 개발·운영하며 기능 구현, 구조 설계, 도구 개발로 문제를 해결한 경험을 정리했습니다.

제품 요구사항을 구현하고 서비스를 운영하며 마주한 상태 관리와 의존성, 화면 전환, 플랫폼 확장 문제를 풀어왔습니다. 기술 선택은 해결할 문제와 프로젝트 상황을 기준으로 판단했으며, 그 경험을 라이브러리 개발과 오픈소스 기여로 확장했습니다. 직접 만든 windowinsets.info와 Dari는 각각 **Android Weekly #747·#748**에 소개되었습니다.

Side Projects

YeoBee(여비) - 여행 비용 기록과 정산을 돕는 여행 가계부 2026.01 ~



여행 동행자와 함께 지출을 기록하고 공동경비·개인경비·정산 내역을 관리하는 서비스

Android 개발 · Play Store

- **Metro 전환**: 기존 Hilt의 graph·scope·qualifier 기반 의존 구조를 유지하면서 KSP/KAPT 코드 생성 경로를 제거하기 위해, Kotlin compiler plugin의 FIR/IR에서 직접 코드를 생성하는 Metro를 선택하고 DI 설정 재정의
- **Navigation3 도입**: 딥링크·앱 내부 화면 전환·비로그인·warm start마다 달라지는 stack 구성을 하나의 상태에서 관리하기 위해, back stack을 앱 상태로 직접 다루는 Navigation3를 선택. 진입 경로별 stack 구성과 초대 수락·오류 처리 흐름 일원화
- **초대 수락 흐름 재설계**: 딥링크·코드 입력·비로그인·warm start에서도 초대 만료·중복 참여·정원 마감·여행 삭제를 같은 규칙으로 검증하고, 이름 선택이 끝난 시점에만 최종 수락 API를 호출하도록 구성해 중복·잘못된 참여 요청 방지
- **AI 에이전트 중심 개발 워크플로 설계**: Figma MCP, Task·Lesson 기록, AI가 참조할 개발 문서, `start-workflow` 스킬과 PR Review Bot을 연결해 이슈 생성부터 구현·검증·PR까지 자동화. AI 에이전트가 일관된 기준으로 작업할 수 있도록 요구사항 분해·컨텍스트 설계·변경 리뷰·실행 검증 체계 구축
- **배포 자동화**: QA 배포와 Play Store 운영 배포 때마다 빌드·업로드·versionCode·알림 단계를 반복해야 했던 흐름을 줄이기 위해 Firebase App Distribution 배포 스크립트와 versionCode override를 포함한 GitHub Actions workflow를 구성하고 Discord 알림까지 연결

> 배운 점
>
> - **작업 컨텍스트 구조화**: 작업이 길어질수록 AI가 진행 맥락과 완료 조건을 놓치는 문제를 줄이기 위해 할 일을 Task로 나누고, Figma MCP와 개발 문서로 디자인·모듈 구조·상태 관리 규칙을 제공. AI의 작업 진행 능력은 긴 프롬프트보다 필요한 맥락을 적절한 시점에 제공하는 구조에 더 크게 좌우된다는 점을 확인
> - **실패를 외부 기억으로 축적**: 리뷰와 작업 실패의 원인·수정 기준을 Lesson 패턴으로 남기고 이후 Task와 개발 문서에서 다시 참조하도록 구성. 한 번의 수정으로 끝내지 않고 이전 시행착오를 다음 작업의 입력으로 전환
> - **구현·검증 루프 자동화**: 이슈 생성, 브랜치 분리, 구현, 빌드·테스트, 커밋, PR 생성을 `start-workflow` 스킬로 연결하고 PR Review Bot의 피드백을 Lesson과 문서에 반영. 코드 생성뿐 아니라 검증과 피드백까지 닫힌 루프로 만들어야 안정적으로 작업을 위임할 수 있음을 확인

Reed(리드) - 문장과 감정을 함께 담는 독서 기록 2025.05 ~ 2026.04



독서 중 만난 문장과 감정을 함께 기록하고 공유하는 서비스

Android 개발 · YAPP 26기 최우수상 · GitHub · 기술 기록

- **Circuit 도입**: 여러 화면의 상태 생성과 이벤트 처리를 동일한 계약으로 관리하기 위해 Presenter와 UI의 역할을 프레임워크 수준에서 분리하는 Circuit(MVI)을 선택. Presenter는 상태 생성·이벤트 처리를, UI는 상태 소비·이벤트 전달을 담당하도록 해 화면 구조 표준화
- **기술 도입 리딩**: Circuit 적용 전 기대 효과와 제약, 일회성 이벤트 처리의 우려를 팀에 공유하고 Presenter·UI·Screen의 책임 규칙을 함께 합의해 구조 전환의 기준 정립
- **타입 기반 화면 전환**: 기록·감정 데이터와 동적 복귀 경로를 안전하게 함께 전달하기 위해 Screen 기반 Circuit Navigation을 선택. 화면 인자를 타입으로 정의해 route·custom `NavType` 보일러플레이트 제거
- **Pagination 적용**: 검색 결과 전체를 기다리는 초기 대기를 줄이기 위해 페이지 단위 요청 방식을 구현하고 첫 페이지부터 화면에 표시
- **로그인 전 탐색 지원**: 가입 전에 서비스의 기록 방식을 확인할 수 있도록 Guest Mode로 주요 기능 체험 경로 제공
- **기록 카드 공유**: Compose UI를 이미지로 바로 공유할 수 없어 `GraphicsLayer`로 화면을 `Bitmap`으로 변환하고, 완성한 도서 기록 카드의 저장·공유 지원
- **OCR 문장 입력**: 책 속 문장을 직접 옮겨 적는 부담을 줄이기 위해 촬영 이미지를 Google Cloud Vision API로 텍스트로 변환하고, 인식 결과를 기록 템플릿에서 바로 편집·저장할 수 있도록 연결

화면 이탈로 취소된 코루틴을 API 실패로 노출한 문제 해결

- **문제**: 도서 항목을 동시에 눌러 상세 화면이 중복으로 쌓인 뒤 빠르게 뒤로 가면, 내 서재 화면에 코루틴 오류가 Toast로 노출됨
- **원인 분석**: 상세 화면이 연속으로 pop되면서 `LaunchedEffect`의 scope와 진행 중인 API 요청이 취소됨. 이때 발생한 `CancellationException`을 `catch (Throwable)`이 일반 통신 오류로 처리해 사용자에게 실패 메시지를 표시한 사실을 확인
- **해결**: `CancellationException`은 다시 throw해 상위 scope로 취소 신호를 전달하고, 나머지 예외만 공통 오류 처리기로 전달. 서로 다른 항목을 동시에 누를 수 없도록 멀티 터치도 차단해 상세 화면의 중복 적재 방지
- **결과**: 화면 생명주기에 따른 정상 취소와 실제 API 실패를 분리하고, 빠른 연속 조작에서도 잘못된 오류 메시지와 중복 화면 전환이 발생하지 않도록 개선

기술 블로그

Hilt → Metro 마이그레이션 컴파일 오류 해결

- **문제**: 운영 중인 멀티 모듈 프로젝트를 Hilt에서 Metro로 전환하던 중, 문제 위치나 누락된 타입을 알려주지 않는 `IrErrorTypeImpl` 컴파일 오류가 발생해 마이그레이션이 중단됨
- **원인 분석**: 서명 키 없이 누구나 빌드할 수 있는 **최소 재현 프로젝트**를 구성하고 Metro Maintainer와 원인을 추적. Hilt는 각 모듈의 클래스패스를 개별 처리하지만 Metro는 앱 모듈에서 전체 의존성 그래프를 분석한다는 차이를 바탕으로, public API에 노출된 `Preferences` 타입이 `implementation` 의존성에 가려진 것을 확인
- **해결**: `DataStore<Preferences>`를 외부에 노출하는 모듈의 의존성을 **`implementation`에서 `api`로 변경**해 앱 모듈의 컴파일 클래스패스에 타입을 전이하고, 모듈 간 public API 경계를 명확히 정리
- **결과**: 컴파일 오류를 해결하고 **Hilt → Metro 마이그레이션 PR을 완료**. 재현 환경과 분석 내용을 공개해 동일한 오류를 추적할 수 있는 근거 제공

기술 블로그 · Maintainer 논의 · Migration PR

> 배운 점
>
> - **일회성 이벤트 모델링**: 이벤트 유실을 피하려 State로 관리했지만, 소비 상태를 초기화하지 않아 화면 복원 시 같은 스낵바가 다시 노출되는 문제를 겪음. 이후 자료형보다 이벤트의 유실·소비·복원 시점을 먼저 정의하는 기준을 세움
> - **화면 이탈과 코루틴 취소**: 화면 이탈로 Presenter의 작업이 취소되는 것은 실패가 아니라 생명주기에 따른 정상 종료일 수 있으므로, `CancellationException`을 일반 오류와 구분해 정상 취소와 실제 실패를 나누는 오류 처리 기준 정립
> - **모듈 간 의존성 경계**: Hilt에서 Metro로 전환하던 중 public API가 DataStore 타입을 노출하고 있는데도 해당 의존성을 `implementation`으로 선언해 컴파일러가 타입을 해석하지 못하는 문제를 겪음. 최소 재현 프로젝트로 원인을 좁히며 `api`와 `implementation`의 선택 기준은 관례가 아니라 public API의 타입 노출 여부임을 확인

유니페스 : 대학 축제의 지도를 펼쳐라! 2024.03 ~ 2025.10



대학 축제 통합 플랫폼으로 지도 기반 행사 정보, 부스 웨이팅, QR 인증 이벤트, 알림 기능을 제공하는 서비스

Android 개발 · Play Store · GitHub · 기술 기록

고려대·가천대·상명대·한국교통대 축제 공식 앱 선정, Play Store 다운로드 **2,000+**, 축제 운영 기간 Android/iOS 통합 최고 **WAU 5,000+**

- **지도 클러스터링**: 좁은 캠퍼스에 밀집된 부스·행사 마커를 구분하기 위해, 공식 지원이 막 시작되어 Compose 레퍼런스가 샘플 코드뿐이던 NaverMap Compose 클러스터링을 도입. 모든 부스가 하나로 뭉치지 않도록 줌 레벨별·카테고리별 병합 전략을 직접 설계하고, 클러스터를 누르면 묶인 부스 목록을 보여주도록 구현 (기술 블로그)
- **MVI 기반 모듈화**: 축제 기능별 상태와 이벤트를 분리해 관리하기 위해 구글 권장 아키텍처와 MVI 패턴을 선택. 화면 상태와 이벤트 처리 책임을 기능 모듈 안으로 분리
- **Type-safe 중첩 Navigation**: 문자열 route와 argument key 불일치를 줄이기 위해 sealed route 기반 Navigation으로 전환하고, 중첩 graph에서도 data class property와 `SavedStateHandle` key를 연결해 부스 상세·위치 화면의 인자 전달 규칙 정리
- **QR 기반 현장 인증**: 부스의 QR 스캔 결과를 참여 상태에 연결해 현장 이벤트 인증을 앱 안에서 완료
- **탐색 상태 복원**: 앱 재실행 후에도 관심 축제·부스와 사용자 설정을 유지하도록 Room과 DataStore에 데이터 성격별로 저장
- **운영 중 버전·예외 대응**: 축제 기간에 지원 버전 기준을 새 빌드 없이 바꿀 수 있도록 Firebase Remote Config로 원격 관리하고, API 에러 코드를 상황별 안내 흐름에 연결
- **Room Migration Test**: 스키마 변경 후에도 기존 관심 축제·부스 데이터가 유지되는지 확인하기 위해, 이전 스키마에서 현재 버전까지의 마이그레이션 테스트 구성

웨이팅 신청 버튼의 상태가 갱신되지 않는 문제 해결

- **문제**: 웨이팅 인원·전화번호·개인정보 동의 조건을 모두 충족해도 신청 버튼이 활성화되지 않아 현장 웨이팅을 등록할 수 없었음
- **원인 분석**: `derivedStateOf` 내부의 `partySize`와 `isPrivacyPolicyClicked`는 Compose State가 아닌 일반 변수였음. `remember` key에도 두 값을 지정하지 않아 Snapshot 시스템이 변경을 추적하지 못하고 초기 값으로 유효성을 계속 계산한 사실을 확인
- **해결**: 일반 변수인 인원 수와 동의 여부를 `remember` key로 명시하고, Snapshot State인 `TextFieldState.text`는 `derivedStateOf` 내부에서 관찰하도록 의존성 구성 수정
- **결과**: 입력 조건에 맞춰 신청 버튼이 갱신되도록 복구하고, 전화번호 입력마다 버튼을 사용하는 UI 전체가 불필요하게 재구성되지 않도록 검증 로직 정리

기술 블로그

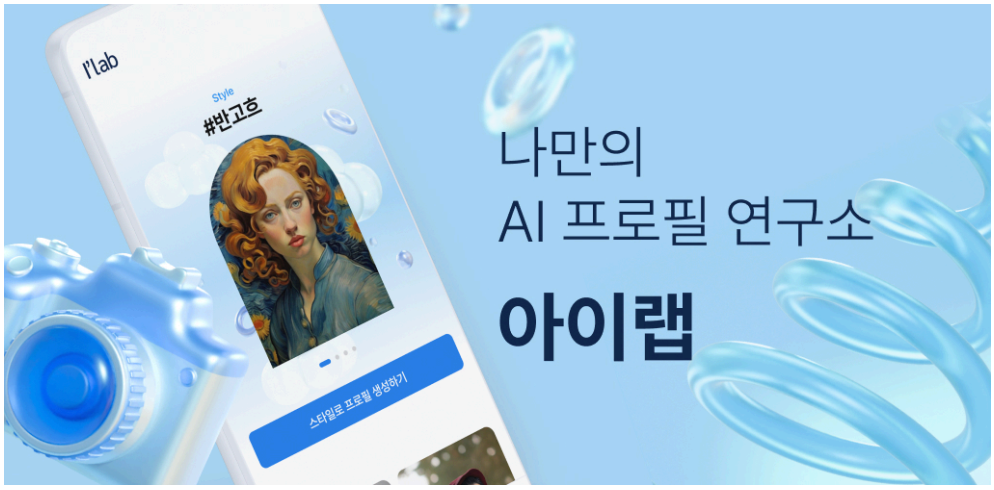
동적 이미지와 겹치는 시스템 아이콘 가독성 개선

- **문제**: 부스 상세 이미지가 StatusBar와 TopAppBar 영역까지 채워지면서, 이미지 밝기에 따라 상태 표시줄과 뒤로가기 아이콘이 배경에 묻혀 보이지 않음
- **원인 분석**: 실제 배경은 매번 달라지는 네트워크 이미지인데 아이콘 색상은 라이트·다크 테마만 기준으로 결정하고 있었음. 이미지 휘도를 계산하는 초기 구현도 화면에 사용하지 않는 영역을 포함해 모든 픽셀을 순회
- **해결**: Coil로 불러온 `Bitmap`의 RGB 값에 BT.709 가중치를 적용해 휘도를 계산. 아이콘과 겹치는 이미지 상단 30%에서 가로 약 50개·세로 약 20개 지점을 샘플링하고, CPU 연산은 `Dispatchers.Default`에서 처리해 밝기에 따라 대조되는 아이콘 색상 적용
- **결과**: 라이트·다크 테마와 별개로 동적 이미지의 밝기에 맞춰 아이콘 가독성을 개선하고, 이미지 전체 픽셀을 순회하던 계산 범위 축소

기술 블로그

> 배운 점
>
> - 신규 기능을 배포하며 변경된 코드의 동작뿐 아니라 이전 버전에서 쌓인 사용자 데이터가 새 스키마에서도 유지되는지 함께 검증하는 것을 배포 원칙으로 삼음
> - 로그인 없이 SSAID로 사용자를 식별해 진입 장벽을 낮췄지만, 동일 기기에서도 앱 서명 키가 달라지면 값이 바뀌어 개발 머신별 QA 데이터가 분리되는 문제를 겪음. 플랫폼 식별자는 반환값만 믿지 않고 빌드 변형·서명·재설치 조건까지 확인해야 함을 배움
> - `derivedStateOf`를 막연히 재구성 최적화 도구로 사용했다가 일반 변수가 Snapshot 추적 대상이 아니어서 웨이팅 버튼이 갱신되지 않는 문제를 만남. 최적화 API를 적용하기 전에 어떤 상태가 어디서 관찰되고 언제 다시 계산되는지 먼저 확인하게 됨
> - **콘텐츠 기반 UI 판단**: 테마 같은 간접 상태만 사용하지 않고 실제 배경 이미지의 밝기를 아이콘 색상 결정에 반영. 비용이 큰 연산은 문제와 직접 관련된 상단 영역과 대표 표본으로 제한하며, 최적화 전에 필요한 입력 범위와 정밀도를 먼저 정의하게 됨

I'Lab - 나만의 AI 프로필 연구소 2024.01 ~ 2024.04



생성형 AI 기반으로 취향에 맞는 프로필 사진을 만들고 공유할 수 있는 카메라 앱

Android 개발 · Play Store · GitHub

- **Orbit MVI 도입**: 이미지 선택·스타일 적용·생성·저장으로 이어지는 비동기 과정의 상태와 일회성 이벤트를 구분하기 위해 Orbit 기반 MVI를 선택. Orbit Container에서 상태와 사이드 이펙트를 분리하고 Clean Architecture 계층과 연결
- **빌드 설정 공통화**: 모듈을 추가할 때마다 플러그인·Android·의존성 설정을 반복해야 했던 지점을 줄이기 위해 Version Catalog와 Gradle Convention Plugin을 도입. 공통 설정을 빌드 로직으로 이동해 중복 감소
- **프로필 제작 흐름 연결**: 카메라·앨범의 사진 선택부터 스타일 적용, AI 이미지 생성, 결과 저장·공유까지 하나의 흐름으로 연결

> 배운 점
>
> - 당시 주목받던 Orbit으로 MVI를 구현하며 상태와 일회성 이벤트를 구분하는 기준을 익혔지만, MVI의 핵심은 특정 프레임워크보다 팀이 합의한 상태·이벤트 처리 규칙을 일관되게 지키는 데 있음을 깨달음. 컨벤션이 명확하다면 전용 MVI 프레임워크는 필수가 아니며, 라이브러리의 편의와 프로젝트 규모·팀 숙련도·추가 의존성을 함께 비교해야 함
> - ComposeInvestigator가 ProGuard를 적용한 Release 빌드에서 문제를 일으켜 비활성화한 경험을 통해, 개발 중 유용한 분석 도구도 실제 배포 경로와 난독화 환경까지 확인한 뒤 도입해야 함을 배움

반다라트 - 부담 없는 만다라트 계획표 2023.07 ~



기존 9x9 만다라트 계획표를 모바일 환경에 맞게 5x5 구조로 줄인 목표 관리 앱

Kotlin Multiplatform 개발 · Play Store · App Store · GitHub · 다운로드 **500+**

iOS 출시기 · Koin·expect/actual 기반 네이티브 기능 전환기

- **Compose Multiplatform 전환**: 기존 Android Compose 코드의 재사용 범위를 넓혀 iOS로 확장하기 위해 Compose Multiplatform을 선택. 공통 UI를 구성해 Android 앱을 유지하면서 iOS 앱까지 배포
- **offline-first 전환**: 서버 운영 중단 뒤에도 사용자가 작성한 목표를 계속 조회·편집할 수 있어야 했기 때문에 서버 API 의존 구조를 Room 로컬 저장소 중심으로 바꾸고 서비스 유지
- **모바일 계획표 UI**: 9x9 표를 작은 화면에 그대로 옮겼을 때 생기는 탐색 부담을 줄이기 위해 핵심 목표와 하위 목표를 5x5로 재구성하고, Jetpack Compose Custom UI와 공통 컴포넌트로 계획표 조작 방식 통일
- **계획표 상태 관리**: 목표 조회·편집·완료 과정의 상태 생성과 이벤트 처리가 UI에 섞이지 않도록 기존 MVVM 구조를 Circuit 기반 MVI로 전환하고 Presenter와 UI의 책임 분리
- **업데이트 경로 제공**: 새 버전이 배포돼도 사용자가 스토어에서 업데이트 여부를 따로 확인하지 않도록 Google Play In-App Update API로 앱 안에서 새 버전을 안내하고 업데이트하는 흐름 구성
- **CI 도입**: Room Database·Repository·ViewModel 변경 때마다 주요 데이터·화면 로직을 반복 검증해야 했던 과정을 자동화하기 위해 테스트 코드를 GitHub Actions에서 실행하도록 구성
- **넓은 화면 대응**: 세로 휴대폰 기준으로 고정돼 있던 화면을 Galaxy Z Fold·iPhone Duo·iPad·휴대폰 가로 화면까지 대응. 기기 종류 대신 콘텐츠 영역의 가로세로 비율로 분기해(너비가 높이의 0.9배 이상이면 목표 정보와 계획표를 좌우로 나란히 배치) 한 기준으로 모든 기기를 처리하고, 표 크기에 따라 셀 글자 배율을 0.85~1.5배로 조절. 공유 이미지도 2단 화면에서는 가로형으로 만들고, iPad에서 공유 시트 위치를 지정하지 않으면 앱이 종료되는 문제를 막음

KMP 이미지 저장·공유의 플랫폼 경계와 iOS 색상 왜곡 해결

- **문제**: Android `Context` · `MediaStore` · `FileProvider` 에 의존하던 이미지 저장·공유 코드를 `commonMain` 에서 사용할 수 없었고, iOS에서 `ImageBitmap` 을 `UIImage` 로 변환한 초기 구현은 이미지 색상이 바래짐
- **원인 분석**: Android와 iOS는 네이티브 API와 URI 타입이 다르며, `ImageBitmap` 과 `UIImage` 도 픽셀·색 공간 처리 방식이 달라 하나의 구현을 그대로 공유할 수 없었음
- **해결**: 공통 코드에는 `ImageHandlerProvider` 계약을 두고 Koin의 플랫폼별 모듈에서 `actual` 구현을 제공. Android는 `Application` 을 주입해 `MediaStore` · `FileProvider` 를 사용하고, iOS는 sRGB 색 공간과 알파·바이트 순서를 지정한 CoreGraphics 컨텍스트에서 `UIImage` 로 변환
- **결과**: 공통 UI가 동일한 계약으로 Android·iOS의 이미지 저장과 공유를 호출하도록 구성하고, iOS 변환 과정의 색상 왜곡 해결

기술 블로그 · iOS 출시기

> 배운 점
>
> - 이미지 저장·공유처럼 플랫폼 API가 필요한 기능은 `expect/actual` 과 플랫폼별 Koin 모듈로 분리해 공통 코드가 Android `Context` 나 iOS 타입을 직접 알지 않도록 구성함. 이 과정에서 KMP 전환은 Android 코드를 `commonMain` 으로 최대한 옮기는 작업이 아니라, 공통으로 유지할 책임과 플랫폼에 남길 책임의 경계를 먼저 정하는 것을 전환 기준으로 삼음
> - iOS `actual` 구현에서 Kotlin/Native로 UIKit·Foundation·CoreGraphics를 직접 다루고 `ImageBitmap` 을 `UIImage` 로 변환하는 과정에서 픽셀 배치·색 공간·알파 채널·바이트 순서에 따라 색상이 변하는 문제를 겪음. 이를 해결하며 KMP가 플랫폼 코드를 줄여주더라도 플랫폼별 이미지 처리 원리와 네이티브 동작까지 함께 이해해야 한다는 점을 체감
> - Circuit과 Koin이 함께 동작하지 않는다고 이슈와 문서 검색만으로 판단해 다른 구조로 우회했지만, 이후 실제 연동 방법이 있음을 확인함. 레퍼런스가 적다는 이유만으로 호환 여부를 단정하지 않고 최소 구현으로 직접 검증한 뒤 기술을 선택해야 함을 배움

Other Side Projects

이끔 2023.07 ~ 2024.04



카페에서 공부하는 사용자를 위한 맞춤 카페 탐색 앱

Android 개발 · GitHub · 기술 기록

- **지도·목록 상태 동기화**: 지도에서 선택한 카페와 목록에서 강조되는 항목이 달라지지 않도록 검색 결과와 Naver Map 마커가 같은 선택 상태를 사용하게 해 두 탐색 방식의 맥락 유지
- **검색 결과 최신성**: 조건이 바뀐 뒤 늦게 도착한 이전 응답이 최신 화면을 덮어쓰지 않도록 검색어 디바운스와 Flow `flatMapLatest`로 이전 검색 작업 취소

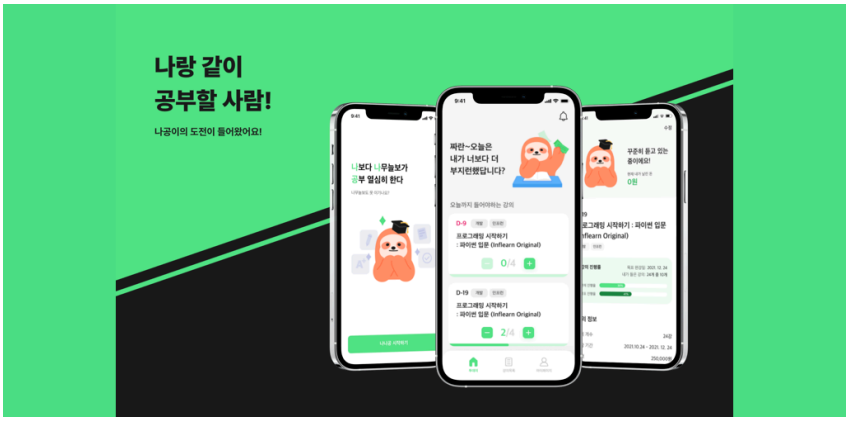
Feature 모듈 간 화면 전환의 순환 의존성 해결

- **문제**: Login 모듈이 로그인 후 Main·Onboarding 화면을 참조하는 상태에서, Main 모듈의 로그아웃 기능이 Login 화면을 다시 참조해 `Login ↔ Main` 순환 의존성이 발생하고 모듈 관계를 구성할 수 없었음
- **원인 분석**: Activity 기반 화면 전환을 위해 각 Feature 모듈이 이동할 화면의 구현 모듈을 직접 참조하고 있었음. Setting 모듈을 추가로 분리해도 화면 전환의 의존 방향이 그대로여서 순환 구조가 해소되지 않음을 확인
- **해결**: 화면 전환 인터페이스를 별도 Navigator 모듈로 분리하고 모든 Feature 모듈이 구현 대신 계약만 의존하도록 변경. 목적지 Feature 모듈에서 Navigator 구현체를 만들고 Hilt로 바인딩해 호출 모듈에 주입
- **결과**: Feature 모듈 간 직접 참조를 제거해 빌드 순환 의존성을 해소하고, 로그인·온보딩·메인 화면 전환을 공통 계약으로 통일

기술 블로그

> 배운 점
>
> - Compose 중심 개발 중에도 기존 View/XML 구현 감각을 잃지 않기 위해 의도적으로 XML 기반으로 제작함. 최신 기술만 반복해 쓰기보다 기존 코드베이스를 유지보수할 수 있도록 기술 범위를 유지하는 것도 필요하다고 판단함
> - 검색 조건이 바뀐 뒤 늦게 도착한 이전 응답이 최신 화면을 덮어쓸 수 있어 `flatMapLatest`로 이전 작업을 취소함. 비동기 검색은 요청의 성공 여부뿐 아니라 어떤 결과가 현재 사용자 입력에 유효한지도 관리해야 함을 배움
> - **모듈 간 화면 전환 경계**: Feature 모듈을 더 잘게 나누는 것만으로는 화면 전환의 순환 의존성이 사라지지 않았음. 화면 구현 참조를 Navigator 계약으로 분리하며, 모듈화에서는 파일의 위치보다 의존 방향과 변경 전파 범위를 먼저 설계해야 함을 배움

나나공 2021.09 ~ 2023.04



인강 수강 독려 서비스 앱

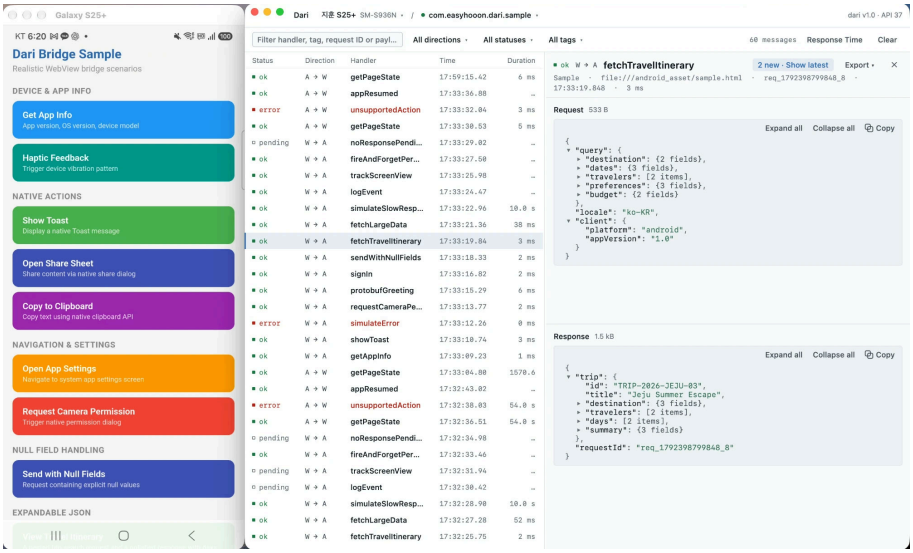
Android 개발 · GitHub · 트러블슈팅 기록

- **수강 독려 흐름**: 사용자가 정한 인강 계획을 놓치지 않도록 FCM 기반 주기적 알림을 제공하고, 알림에서 수강 화면으로 이어지는 진입 경로 구성
- **상태 흐름 전환**: 알림과 수강 상태를 화면마다 별도로 갱신하지 않도록 LiveData 기반 흐름을 Flow 스트림으로 전환해 각 화면이 동일한 상태를 구독하도록 구성

> 배운 점
>
> - 1년 3개월 동안 서비스를 운영·리팩토링하며 수동 Fragment 전환을 Jetpack Navigation으로, LiveData를 StateFlow로, SharedPreferences를 DataStore로 단계적으로 바꾸는 과정에서 Android 개발 방식의 변화를 직접 경험함
> - 기존 수강·알림 흐름을 유지한 채 화면과 상태 관리 책임을 단계적으로 옮기며, 운영 중인 제품의 리팩토링은 새 기술 도입보다 기존 동작을 깨뜨리지 않는 전환 순서가 중요함을 배움

Libraries

Dari - WebView Bridge Inspector



Web ↔ App JavaScript Bridge 메시지의 요청·응답·상태를 기기에서 실시간으로 확인하는 Android 디버깅 라이브러리

GitHub · 개발 배경 · 운영 적용 및 고도화

- **개발 배경**: HTTP 통신은 Chucker로 요청과 응답을 쉽게 확인할 수 있지만, 표준화된 도구가 없는 WebView Bridge는 Logcat에서 흩어진 로그를 직접 찾고 짝지어야 했음. 호출이 빈번하거나 IDE를 연결할 수 없는 환경에서는 **문제 재현이 어렵고, 프론트엔드 개발자와 실패 지점을 공유하기도 힘든 상황**이 반복됨
- **핵심 구현**: 브릿지 레이어에 Interceptor를 연결해 **Web-to-App/App-to-Web 메시지를 함께 수집**하고, 핸들러·상태별 검색과 필터링, Request/Response 매칭, JSON 포매팅, 복사·공유 기능을 제공. Persistent Notification과 Dynamic Shortcut으로 앱 밖에서도 최근 통신과 디버그 패널에 접근하도록 구성
- **운영 검증과 고도화**: 실제 운영 서비스의 개발 환경에 적용한 뒤 인메모리 로그 유실, 단방향 이벤트, 다중 WebView 로그 충돌, 대용량 payload 공유 시 `TransactionTooLargeException`을 발견. **인메모리 `StateFlow` 즉시 반영과 Room 비동기 저장, 최대 건수·TTL 기반 로그 정리, optional request id, 브릿지별 tag, payload 길이 제한**을 도입해 해결
- **사용성 개선**: 전체 로그 JSON 내보내기, `Error` · `In Progress` · `Success` 메시지 상태 필터, 다크모드, shake-to-open을 추가하고 흔들기 감지를 방향 반전 카운팅 방식으로 바꿔 오탐 완화
- **프로덕션 안전성**: Debug 빌드에서는 통신을 기록하고 Release 빌드에서는 동일한 API의 **`dari-noop` 모듈로 교체**해 로그 노출과 런타임 오버헤드를 방지
- **Dari Desktop**: 기기 화면에서 긴 JSON을 읽기 어려운 문제를 줄이기 위해, adb만으로 연결 기기의 브릿지 메시지를 PC에서 요청·응답 나란히 보여주는 Compose Desktop 앱 개발. 토큰·비밀번호·쿠키는 기기에서 마스킹한 뒤 전송하고, macOS 서명·공증과 앱 내 업데이트, Windows 배포와 Linux 프리뷰 배포까지 구성
- **Gradle 플러그인으로 수동 호출 제거**: Chucker는 OkHttp Interceptor 한 줄로 모든 HTTP 통신을 기록하지만, 웹뷰 브릿지는 표준 프로토콜과 단일 진입점이 없어 호출 지점마다 Interceptor 코드를 직접 넣어야 했음(#4)
  - 대안 검토: KSP는 새 파일만 생성해 기존 `@JavascriptInterface` 본문을 고칠 수 없고, Kotlin IR 컴파일러 플러그인은 Java 브릿지와 AAR 안 클래스를 놓치며 Kotlin 버전마다 컴파일러 내부 API를 따라가야 하고, `java.lang.reflect.Proxy`는 WebView가 실제 클래스에 선언된 `@JavascriptInterface` 메서드만 노출해 동작하지 않는다는 근거로 배제
  - 스파이크: AGP Instrumentation API(`AsmClassVisitorFactory`)로 매핑 없이 모든 `@JavascriptInterface` 메서드와 `evaluateJavascript` 호출을 잡아 보니, 실기기에서 요청 10건과 응답 7건이 모두 기록됐지만 핸들러가 `BridgeInterface.onBridgeRequest`로, 응답이 `javascript:onBridgeResponse(…)` 스크립트 문자열로 남고 `simulateError`도 SUCCESS로 기록돼 읽을 수 없는 데이터가 됨을 확인. 그래서 브릿지 메서드를 `dari { }`에 한 번 매핑하는 방식과 완료 기준(release 빌드 미변경, `dari-noop` debug 빌드 정상 동작, 기본 계측 범위는 프로젝트 클래스)을 정의
  - 구현·검증: debug 빌드에서만 매핑한 메서드와 `addJavascriptInterface` 호출부, `doUpdateVisitedHistory`를 ASM으로 고쳐 같은 `DariInterceptor` 호출을 삽입. 실기기에서 같은 시나리오(페이지 동작 16개와 App→Web 호출)를 돌려 기록 18건의 핸들러·payload·상태가 수동 방식과 같음을 확인했고, 응답이 없는 fire-and-forget 호출은 IN_PROGRESS에 머물지 않고 SUCCESS로 처리. 수동 API는 기본값으로 유지해 기존 사용자는 버전만 올리면 됨(PR #218)
- **사내 도입**: Hi-me·세컨드 윈드·브리시드·마이밸런스 4개 앱의 Debug 빌드에 적용하고, Hi-me는 QA(STG) 빌드에도 포함. 전에는 Logcat에서 요청과 응답을 직접 찾아 짝지어야 했고, 프론트엔드 개발자는 네이티브 로그를 볼 수 없어 실패가 웹 쪽인지 앱 쪽인지 말로 맞춰봐야 했음. 지금은 양쪽 개발자가 기기나 PC 화면에서 같은 요청·응답을 보고 로그를 주고받으며 원인을 찾고, IDE를 연결할 수 없는 QA(STG) 환경의 버그도 테스트 기기에 남은 로그로 추적
- **성과**: Android Weekly #748 Libraries & Code 섹션 소개 (2026.10.11). 반복되는 사내 디버깅 문제를 **재사용 가능한 오픈소스 도구로 전환**하고, 직접 사용하는 서비스에서 발견한 예외 상황을 다시 라이브러리 개선으로 연결

Dari Desktop 배포 파일에서 실행 중 쓰지 않는 코드·라이브러리 걷어내기

- **문제**: macOS(arm64) DMG가 70.7 MB였음. 번들을 열어보니 실행 중 한 번도 로드되지 않는 파일이 상당 부분을 차지. arm64 앱 안에 Intel용 Skiko 네이티브 라이브러리(압축 전 21.7 MB)가 같이 들어 있었고, 축소하지 않은 jar가 35.6 MB
- **원인 분석**: Skiko의 macOS 런타임 jar는 arm64·x64 라이브러리를 모두 담고 있고, Compose Gradle 플러그인은 현재 아키텍처용만 꺼낸 뒤 나머지를 jar에 다시 넣어 배포함을 확인. 최신 Skiko에서도 같은 구조라 버전 업그레이드로는 해결되지 않음

- **해결**: Gradle artifact transform으로 패키징 전에 다른 아키텍처 라이브러리를 제거하고, Compose ProGuard release 빌드로 jar를 축소(로그 스택 트레이스를 읽을 수 있도록 난독화는 끄고 `META-INF/services` 로 로드되는 클래스만 keep). 압축은 2.2.0에서 LZMA(ULMO)로 바꿔 58.1 MB까지 줄였으나, macOS 커널이 바로 풀지 못해 설치·앱 내 업데이트 때 앱 복사가 약 3배 느려지는 것을 측정하고 2.2.1에서 LZFSE(ULFO)로 변경(복사 17.6초 → 5.6초)
- **결과**: 2.2.1 기준 arm64 DMG **70.7 MB → 43.3 MB(38.7% 감소)**, Windows MSI 59.6 MB → 46.1 MB. 이후 기능이 추가된 2.5.2의 arm64 DMG는 45.4 MB. 앱 시작 시간은 측정상 차이가 없었고, ProGuard 빌드의 동작을 macOS는 실사용, Windows는 실기기, Linux는 Ubuntu 가상 머신에서 확인

Skiko 중복 제거 PR · ProGuard 적용 PR · DMG 압축 형식 PR

Dari Desktop의 늘어난 heap이 유휴 상태에서 줄지 않던 문제 해결

- **문제**: 브릿지 메시지가 몰린 뒤 Desktop 앱의 committed Java heap이 약 146 MB까지 커진 채 줄지 않음. 실제로 살아 있는 데이터는 약 20 MB였고, 메시지 요약 2,000건은 약 145 KB에 불과
- **원인 분석**: G1 GC가 메시지가 몰릴 때 늘린 heap을 유휴 상태에서도 OS에 돌려주지 않았음. 또 observe 이벤트가 올 때마다 메시지 맵 전체를 복사하고, 화면이 매 프레임 같은 목록을 세 번씩 다시 계산
- **해결**: G1 주기적 GC(15초)와 작은 초기 heap(`-Xms32m`), `MinHeapFreeRatio` · `MaxHeapFreeRatio` 로 유휴 heap을 반환하게 하고, 채널에 쌓인 이벤트를 한 번에 꺼내 배치당 맵 복사를 1회로 줄임. 파생 목록은 `derivedStateOf` 로 공유하고 메시지 맵에 `referentialEqualityPolicy()` 를 적용해 2,000건 비교를 없앰
- **결과**: 2.7.0과 번갈아 4회씩 같은 조건(메시지 2,500건 수신)으로 측정해 committed heap **128 MB → 34 MB(73% 감소)**, 그래픽을 제외한 앱 메모리 295 MB → 214 MB(27% 감소). Skia 그래픽 메모리는 빌드와 상관없이 실행마다 약 8~155 MB로 흔들려 비교에서 제외

메모리 개선 PR

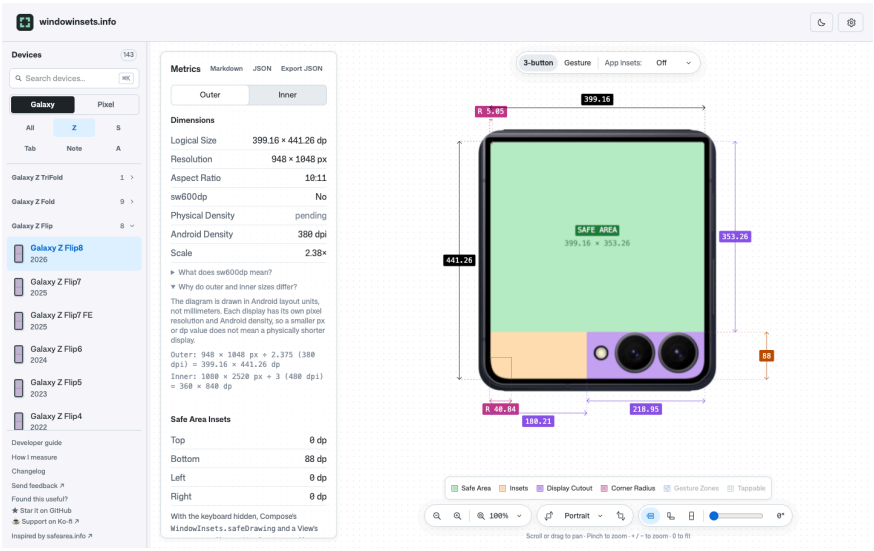
RoutePeek - WebView Route Inspector

WebView의 현재 route와 **SPA route 변경을 Compose overlay로 확인**하는 Android 디버깅 라이브러리. IDE를 연결하지 않아도 웹뷰 화면의 현재 경로를 앱 안에서 확인해 라우팅 이슈 재현 시 경로를 함께 전달 가능

GitHub

Open Source Project

windowinsets.info - Android WindowInsets 측정·시각화 2026.09 ~



Android 기기별 `WindowInsets` , `DisplayCutout` , `RoundedCorner` , `FoldingFeature` 데이터를 측정·시각화하는 오픈소스 도구

Android 측정 앱·데이터 검증·웹 시각화 개발 · 서비스 · GitHub · 개발 기록

- **InsetsProbe 개발**: `WindowInsets` , `DisplayCutout` , `RoundedCorner` , `FoldingFeature` 를 Android 앱에서 읽어 화면·회전 방향·내비게이션 방식·OS 버전과 함께 JSON으로 기록. 창 크기와 실제 화면이 맞지 않거나 회전이 적용되지 않은 캡처는 제외해 잘못된 측정값의 게시 방지
- **측정 과정 자동화**: Samsung Remote Test Lab에서 결과 파일을 수동으로 내려받던 병목을 줄이기 위해, Probe → 업로드 API → GitHub PR로 원본 JSON을 모으는 흐름 구축. Fold8·Flip8 실기기 측정값이 PR에 도착하는 것까지 확인
- **출처가 보이는 데이터 제공**: 실기기·Samsung RTL·Android Emulator 측정값을 구분하고, 확인되지 않은 값은 추정하지 않고 미측정으로 표시. Galaxy와 Pixel의 디스플레이별 `WindowInsets` · `DisplayCutout` 을 디바이스 프리뷰와 수치로 함께 확인할 수 있도록 구성
- **성과**: Android Weekly #747 Libraries & Code 섹션에 **Android Window Insets & Safe Areas**로 소개 (2026.10.04)

Open Source Contributions

아래는 직접 관리하는 Dari·RoutePeek 외에 외부 오픈소스 프로젝트에 병합된 기여입니다.

- android/cahier · Pull Request: Android 공식 Compose 노트 앱 샘플에서 Gson 기반 직렬화를 **kotlinx-serialization으로 마이그레이션**해 컴파일 타임 검증과 타입 안정성 강화
- DroidKaigi/conference-app-2025 · Pull Requests: **5개 PR**로 Navigation3 뒤로가기 처리의 back stack 검증, `enableEdgeToEdge` 기반 SystemBar 구성, `vectorResource` 전환, `FlowRow` 적용
- soil · Pull Request: 샘플 앱에 Type-safe Compose Navigation 적용
- compose-stability-analyzer · Pull Request: README의 Logger 설정 코드와 실제 구현 간 불일치 수정
- Circuit · Pull Request: Circuit Navigation 공식 문서의 변수명 일관성 개선
- DroidKnightsApp · Pull Request: Flow 반환 타입 함수의 불필요한 `suspend` 제거와 관련 테스트 코드 반영

Company Troubleshooting

Hi-me - HD현대 그룹사 임직원 대상 건강관리 헬스케어 서비스 2025.10 ~

Android 개발 · Play Store · 다운로드 **2,000+**

Hi-me를 개발·운영하며 해결한 주요 문제를 원인 분석과 해결 결과 중심으로 정리했습니다.

WebView Renderer 종료 후 사진 업로드 흐름 복구

- **문제**: 특정 기기에서 카메라 촬영 후 앱으로 복귀하면 앱이 재시작된 것처럼 보이고, 촬영한 사진의 업로드 흐름이 중단됨
- **원인 분석**: Crashlytics에 Fatal 로그가 없는 점에서 앱 프로세스와 WebView Renderer 종료를 분리해 추적. `onRenderProcessGone()`에 WebView 버전·URL·가용 메모리·`didCrash`를 Non-Fatal로 수집해 카메라 실행 중 **메모리 압박으로 Renderer가 시스템에 의해 종료**된 사실을 확인
- **해결**: 앱 전체 재시작 로직을 제거하고 WebView를 재생성·복원. 네이티브는 촬영 요청과 결과를 **ViewModel의 pending state**로 보관하고, 웹 측에서는 request id와 진행 상태를 **TTL이 있는** **`localStorage`**에 저장한 뒤 bridge ready 시 결과를 재전달하도록 구현해 네이티브-웹 양쪽 복구 흐름을 맞춤
- **결과**: Renderer가 종료되어 JS 메모리와 callback이 유실되어도 **사용자가 진행하던 사진 요청을 복원해 업로드를 완료**할 수 있는 흐름 구축

기술 블로그

웹뷰 이미지 전달의 메인 스레드 부하 개선

- **문제**: 카메라·갤러리 이미지를 base64 문자열로 JSON에 담아 `evaluateJavascript`로 넘기는 구조여서, 사진 1장마다 수십만 자의 메시지(고정 테스트 사진 기준 58.8만 자)를 메인 스레드에서 직렬화하고 웹에서 다시 파싱함
- **원인 분석**: 개선 전에 이미지 처리·직렬화·`evaluateJavascript` 구간에 Perfetto trace 마커를 넣고 실기기 기준값을 측정. 메인 스레드 점유 14.2ms 중 JSON 직렬화가 10.5ms를 차지해 120Hz 프레임 예산(8.3ms)을 넘는 것을 확인
- **해결**:
  - JSON 직렬화와 JS 명령 문자열 생성을 `Dispatchers.Default`로 옮기고 `evaluateJavascript`만 메인에서 호출. 큰 메시지 뒤에 보낸 작은 메시지가 먼저 도착하지 않도록 `Mutex`로 전송 순서를 유지
  - base64 대신 JPEG를 캐시 파일로 저장하고 웹 페이지와 같은 출처의 로컬 URL만 넘겨 웹이 `fetch`로 받도록 변경. 같은 WebView에 뜬 외부 페이지가 파일을 읽지 못하게 메인 프레임 출처·경로 형식·캐시 폴더 밖 경로를 검사하고, id는 128비트 난수로 발급. 발급 기록을 메모리에 두지 않아 앱 프로세스가 다시 시작돼도 사진 복구 흐름이 동작
  - 웹 코드를 옮긴 테스트 페이지와 Macrobenchmark로 두 방식을 같은 조건에서 비교해 웹 팀에 제안
- **결과**: 실기기 측정(n=5, p50)에서 메인 스레드 점유 **14.2ms → 3.4ms**(-76%). 로컬 URL 방식은 Macrobenchmark(실기기, n=20, p50)에서 웹 File 준비까지 **15.9ms → 5.2ms**(-67%), 브릿지 메시지는 58.8만 자 → 408자. dev·staging에 적용했고 운영 적용은 웹 배포 후 예정

Compose-AndroidView 경계의 터치 이벤트 누수 해결

- **문제**: 스플래시 오버레이 노출 중 터치가 하단 WebView까지 전달되어 JS 클릭 핸들러가 실행됨
- **원인 분석**: `Modifier.clickable`은 Main pass에서 이벤트를 소비하지만, `AndroidViewHolder`의 `pointerInteropFilter`가 Initial pass에서 MotionEvent를 WebView로 먼저 전달하는 구조를 확인
- **해결**: `pointerInput`과 `PointerEventPass.Initial`에서 터치 이벤트를 선점해 consume하는 **`blockTouches()`** **modifier**를 적용
- **결과**: Compose overlay 아래의 WebView로 터치 이벤트가 전달되는 문제를 차단하고, Splash·Scrim·Loading overlay에 재사용 가능한 이벤트 차단 패턴 확보

기술 블로그

하이브리드 앱의 백그라운드 음악 재생 구조 설계

- **문제**: Android WebView가 MediaSession을 지원하지 않고 백그라운드 진입 시 재생을 중단해, 웹 플레이어만으로는 잠금화면·알림 미디어 제어 요구사항을 충족할 수 없었음
- **의사결정**: 일정과 하이브리드 앱의 구조적 일관성을 고려해 전체 화면의 네이티브 전환 대신, **웹은 포그라운드 UI를 맡고 네이티브는 백그라운드 재생을 맡는 역할 분리** 선택
- **해결**: video.js와 Media3 ExoPlayer/MediaSession 사이에 **6개의 양방향 브릿지**를 정의. ExoPlayer 사전 준비와 전환 경과 시간 보정으로 재생 끊김을 줄이고, 양쪽 seek·play·pause 상태를 동기화

- **결과**: 기존 웹 화면을 유지하면서 **백그라운드 재생, 잠금화면·알림 컨트롤, 포그라운드 복귀 시 재생 위치 복원** 지원

기술 블로그

BLE 의료기기 연동과 측정 데이터 동기화 안정화

- **문제**: 혈당계·혈압계 연동에서 OS 본딩 상태와 앱 내 DataStore 연동 정보가 불일치하거나, 기기별 측정·전송 타이밍 차이로 데이터 동기화 실패와 중복 수신 가능성이 발생
- **원인 분석**: Android BLE의 Connection/Bonding 생명주기, GATT 명령 순차 실행 제약, 혈당계 RACP 응답 후 자체 연결 종료, 혈압계 실시간 indication 방식의 차이를 분리해 추적
- **해결**: DataStore의 연동 정보와 OS bondedDevices를 교차 검증하고, 혈당계는 **Report First, Then Delete** 전략으로 측정값을 먼저 확보한 뒤 기기 데이터를 삭제. 혈압계는 `lastSyncDate` 기준 필터링과 로컬 캐싱으로 중복 반영을 방지
- **결과**: Report First, Then Delete와 `lastSyncDate` 기반 필터링·로컬 캐싱을 적용해 측정 데이터 누락·중복과 연동 상태 불일치 위험 완화

기술 블로그

웹 수정 사항을 dev 배포 없이 실기기 웹뷰에서 확인하는 개발 환경 구축

- **문제**: 웹 수정 사항을 앱 웹뷰에서 확인하려면 매번 dev 서버에 배포해야 했고, 확인 중 문제가 나오면 고친 뒤 다시 배포하고 확인함. Jenkins dev 배포는 최근 성공 20건 평균 **2분 30초**(중앙값 2분 20초)
- **원인 분석**: 웹뷰가 개발 PC의 Vite 서버를 열게 하는 데 걸리는 제약을 하나씩 확인
  - PC의 IP 주소로 접속하면 카카오 로그인이 KOE006으로 실패. Redirect URI를 접속 주소 기준으로 검사하므로 `localhost` 를 유지해야 함
  - 앱은 허용 도메인의 페이지에만 브릿지를 열고 cleartext http를 막고 있어, 로컬 서버 페이지에서는 네이티브 기능이 동작하지 않음
  - macOS adb의 기본 mDNS 백엔드(Openscreen)가 사내 Wi-Fi에서 폰의 무선 디버깅 서비스를 찾지 못함
  - 터미널에 문자로 그린 페어링 QR은 줄 간격 때문에 무선 디버깅 스캐너가 읽지 못함
- **해결**:
  - Android에 `local` 빌드를 추가. 패키지명을 분리해 dev 앱과 함께 설치되고, 웹뷰가 `http://localhost:3000` 을 열며 API는 dev 서버를 사용. 브릿지와 cleartext 허용은 local 빌드에서 스킴·호스트·포트가 모두 일치하는 origin 하나로 한정하고 단위 테스트로 고정. GitHub Actions로 Firebase App Distribution에 배포
  - `adb reverse` 로 폰의 `localhost:3000` 을 PC로 넘겨 localhost를 유지. 웹 API는 Vite 프록시를 거쳐 같은 출처로 호출되므로 CORS 설정 변경 없이 동작
  - 웹 저장소에 `pnpm dev-android` 스크립트를 추가해 무선 디버깅 연결, `adb reverse` , Vite 실행을 명령 하나로 처리. adb mDNS를 Bonjour로 고정하고, 페어링 QR을 브라우저 이미지로도 띄우며, QR 대신 페어링 코드로도 연결
- **결과**: 저장 후 실기기 앱 웹뷰에 반영되기까지 평균 **0.46초**(Galaxy S25+, 파일 저장부터 웹뷰 DOM 반영까지 5회 측정, 313~683ms). 카카오·네이버 로그인도 local 앱에서 동작. 스크립트 추가 전후의 dev·staging·production 빌드 결과물이 같은 것을 비교해 배포 영향이 없음을 확인

세컨드 윈드/닥터 - 만성질환자·암 수술 환자 건강관리·재활 서비스 2024.05 ~

Android 개발 · Play Store · 5종 운영 · 전체 출시 앱 누적 다운로드 합산 **13,000+**

테스트가 없던 레거시 Java 6.9만 줄의 97%를 Kotlin으로 전환, 정의한 회귀 케이스에서 동등성 확인

남은 Java 13개 파일(Realm 관련)은 Realm → Room 교체 때 전환 예정

- **문제**: 9개 flavor 앱이 공유하는 Java 69,256줄(194개 파일)은 입사 당시 단위 테스트가 하나도 없었고, 전환 시작 시점에도 그동안 기능·리팩터링 작업에서 직접 작성한 테스트 파일 24개뿐이었음. 건강 데이터·BLE 기기·서버 요청이 얽힌 코드라 언어를 바꾸다 동작이 조금만 달라져도 사용자 데이터에 영향을 줄 수 있었는데, 이를 확인할 방법이 수동 QA뿐이었음
- **검증 원칙**: 전환 전후 동작이 같다는 것을 테스트로 확인한 것만 머지. 리팩터링 중 찾은 기존 버그도 바로 고치지 않고 현재 동작 그대로 테스트에 고정해 옮긴 뒤 이슈로 추적하고, 전환이 끝나면 별도 fix PR에서 테스트 기대값을 바꿔 수정. 동작이 바뀐 이유가 전환 때문인지 버그 수정 때문인지 섞이지 않게 하기 위함
- **A-A 검증**: 파일마다 Java 원본으로 현재 동작과 공개 API 시그니처를 테스트에 먼저 고정(test 커밋)하고 Kotlin으로 옮긴 뒤(refactor 커밋), 스크립트가 test 커밋을 별도 worktree로 꺼내 같은 테스트를 Java·Kotlin 양쪽에서 돌려 비교. 전환 커밋에서 테스트가 바뀌면 실패하도록 해서 테스트를 코드에 맞춰 고치는 일을 막음
- **뮤테이션 테스트**: 테스트가 통과해도 동작 변화를 실제로 잡아내는지 보기 위해 PIT로 코드를 자동 변형하고, `trim` ·Locale·12시간 표기처럼 Java와 Kotlin 표준 함수 차이에서 생기는 변형은 직접 뮤턴트로 추가. 살아남은 뮤턴트는 Java 원본에서 다시 확인한 뒤 테스트를 보강. 수작업 뮤턴트를 PIT 바이트코드 뮤테이션으로 바꿔 뮤턴트당 확인 시간을 약 50초에서 약 1초로 줄이고, 테스트 작성자와 뮤턴트 선택자가 같아서 생기는 편향을 줄임
- **골든 테스트**: 13,792줄짜리 서버 명령 클래스처럼 구조 자체를 바꾸는 경우는 클래스 단위로 비교할 수 없어서, MockWebServer로 정상·HTTP 오류·본문 오류·연결 끊김 등 응답 상황 12종을 재현하고 나가는 요청·콜백 순서·이벤트·저장 상태를 기록한 골든 케이스 **2,924건**과 대조
- **검증 병목 해소**: 테스트가 늘면서 시간 대부분이 검증 대기에 들어, 병렬 worktree 뮤테이션, PIT 실행 순서 변경(재검증 사이클 제거), 단위 테스트 JVM 2분할(전체 실행 23~27분 → 약 14분)을 적용. 화면 테스트는 분기·상태·외부 계약만 고정하는 기준을 세워 힙 부족 해결
- **AI 에이전트 작업 방식**: Claude Code로 정해진 절차는 자동으로 진행하고, 동작이 바뀌거나 되돌리기 어려운 작업만 사람이 확인하도록 설계. 에이전트의 완료 보고는 테스트 통과 수치로만 받도록 검증 규칙 적용

- **결과**: 단위 테스트 **0개 → 1,700개 이상**(A-A 전체 실행 1,703개 통과, 테스트 파일 143개·약 4만 줄)을 작성하며 Kotlin 비율 **12% → 97%**(전환 시작 시점 Java 69,256줄 중 97% 제거, 남은 Java 1,773줄). 1만 줄이 넘던 God class 2개(2024.05 기준 `MainActivity` 18,368줄, API 호출 클래스 `ControlCommand` 14,652줄)를 정리. `MainActivity`는 `NativeCommandHandler`와 함께 책임별 Kotlin 클래스로 분리하고(전환 시작 시점 `MainActivity` 7,499줄 → Kotlin 2,856줄, `NativeCommandHandler` 4,175줄 → Kotlin 167줄), `ControlCommand`는 골든 테스트를 통과한 상태로 정의 표 기반 Kotlin 실행기로 바꿔 전환 시작 시점 13,792줄 → 1,200줄
- **전환 중 찾은 문제**: 동작을 테스트로 고정하다가 알람을 다시 읽으면 연도가 사라지는 문제, 체중계 사용자 ID를 엉뚱한 바이트에서 읽는 문제 등 사용자 데이터에 영향을 줄 수 있는 기존 버그 5건을 찾아 known issue로 등록. 근처 BLE 기기의 광고 이름이 이스케이프 없이 `evaluateJavascript`로 넘어가 웹뷰에서 임의 JS가 실행될 수 있는 보안 취약점도 발견해 `JSONObject` 이스케이프로 수정했고, 같은 패턴이 남은 곳을 정리 중
- **이후 유지보수**: 기능을 고친 뒤 전체 테스트를 돌려 기존 동작이 바뀐 곳이 있는지 바로 확인하고, 남은 Java는 Realm → Room 교체(마이그레이션 테스트로 기존 데이터 읽기를 먼저 고정) 때 옮긴 뒤 known issue로 등록한 기존 버그 5건을 별도 fix PR에서 테스트 기대값을 바꿔 수정할 예정. 이어서 중복 테스트와 전환 이후 필요 없어진 테스트를 지워, 지금도 오래 걸리는 Robolectric·뮤테이션 테스트 실행 시간을 줄일 계획

기술 블로그

Awards

YAPP 26기 최우수상 2026.04
Reed 프로젝트 Android 파트 리드로 구조 설계·기술 스택 검토·코드 리뷰 담당

한국관광공사 X 카카오 2024 관광데이터 활용 공모전 2024.11
트립메이트 Android 앱 개발 및 출시, **장려상 및 강원관광재단 특별상 수상**

Certificates

- 정보처리기사 2022.09

Education

- 건국대학교 컴퓨터 공학부 학사 졸업 2024.02